Artificial Intelligence Review (2025) 58:130
https://doi.org/10.1007/s10462-024-11045-1

Neural combinatorial optimization with reinforcement learning in industrial engineering: a survey

K. T. Chung[1] · **C. K. M. Lee**[1] · **Y. P. Tsang**[1]

Accepted: 29 November 2024 / Published online: 14 February 2025
© The Author(s) 2025

Abstract
In recent trends, machine learning is widely used to support decision-making in various domains and industrial operations. Because of the increasing complexity of modern industries, industrial engineering aims not only to increase cost-effectiveness and productivity but also to consider sustainability, resilience, and human centricity, resulting in many-objective, constrained, and stochastic operations research. Based on the above stringent requirements, combinatorial optimization (CO) problems are thus developed to support the complicated decision-making process in operations research. Due to the computational complexity of exact algorithms and the uncertain solution quality of heuristic methods, there is a growing trend to leverage the power of machine learning in solving CO problems, known as neural combinatorial optimization (NCO), where reinforcement learning (RL) is the core to achieve the sequential decision support. This survey study provides a comprehensive investigation of the theories and recent advancements in applying RL to solve hard CO problems, such as vehicle routing, bin packing, assignment, scheduling, and planning problems, and, in addition, summarizes the applications of neural combinatorial optimization with reinforcement learning (NCO-RL). The detailed review found that although the research domain of NCO-RL is still under-explored, its research potential has been proven to address environmental sustainability, adaptability, and human factors. Last but not least, the technical challenges and opportunities of the NCO-RL to embrace the industry 5.0 paradigm are discussed.

Keywords Neural combinatorial optimization · Reinforcement learning · Industrial engineering · Operations research · Review

✉ C. K. M. Lee
ckm.lee@polyu.edu.hk

K. T. Chung
kwoktung.chung@connect.polyu.hk

Y. P. Tsang
yungpo.tsang@polyu.edu.hk

[1] Department of Industrial and Systems Engineering, The Hong Kong Polytechnic University, Hung Hom, Hong Kong

1 Introduction

Industrial engineering is concerned with the integrated system of people, materials, information, equipment, and energy to predict, evaluate, and improve the performance of the processes and different stages of the systems (Salvendy 2001). Decision-making is a fundamental part of industrial engineering applications, ranging from investment decisions, layout design, production scheduling, inventory control, and routing problems in various domains, including manufacturing, logistics, and supply chain management (Sgarbossa et al. 2020; Triantaphyllou and Mann 1995). Because of the increasing complexity of modern industries, industrial engineering aims not only to enhance cost-effectiveness and productivity but also to consider sustainability, adaptability, and human factors to achieve a broad range of objectives and operational requirements (Colabianchi et al. 2021; Kadir et al. 2019; Manavalan and Jayakrishna 2019; Sgarbossa et al. 2020). In addition, facing the challenges in today's competitive markets, modern manufacturing has evolved to adopt the demand dynamics and unexpected disruptions for timely response. Since conventional static models are inadequate in these areas, dynamic models have been studied to capture the relationships among variables better (Mittal et al. 2008).

When there is a need to model and optimize complex decision-making problems, operations research provides a wide range of methods and techniques for decision-makers to reliably determine optimal solutions for specific operational problems (Shannon et al. 1980). Typically, operations research focuses on the cost minimization of existing processes and formulating decision intelligence (Bengio et al. 2021; Dekker et al. 2012). This paper primarily focuses on combinatorial optimization (CO), which is essential in planning and decision-making in sizeable discrete configuration space to optimize objectives such as minimal cost, maximal payoff, or other performance indicators under different constraints.

In general, CO problems can be solved by exact methods, such as branch-and-bound (B&B) using a mixed-integer linear programming (MILP) formulation (Clausen 1999), but the running time grows exponentially as the problem size increases. No exact algorithm exists to solve the hard CO problems (i.e., NP-hard and NP-complete) within a polynomial time (Korte et al. 2011). Solving large-scale problem instances with an exact solver is impractical, especially when computational resources and decision-making time are limited. Although heuristic methods can solve CO problems in a polynomial time, the solutions have no guarantee of global optimality, which is regarded as a well-known dilemma illustrated by the No Free Lunch Theorem (Wolpert and Macready 1997). Due to the complexity of CO problems and new requirements arising from industry application domains, an effective algorithm to solve CO problems has remained an active research area over the past years. Because of the above challenges, a growing trend is to adopt machine learning to provide a better trade-off between solution quality and execution time. Once the parameterized models are trained on real-world datasets, the results can be generated in polynomial time. The potential capabilities of these models are vast, e.g., forecasting demands and predicting vulnerabilities, which can be further used as variables to support decision-making (Mittal and Panchal 2023).

Neural combinatorial optimization (NCO), which originated from Hopfield and Tank (1985), demonstrates the use of a Hopfield neural network to solve classical CO problems, namely the traveling salesman problem (TSP). Over the past decade, NCO has become a more promising research field motivated by the rapid advancements of machine learning

algorithms and the enabling hardware developments (Bello et al. 2016; Smith 1999). NCO showed promising performance in balancing solution quality and running time, compared with heuristic and exact algorithms. However, the lack of optimal solutions for training data becomes the primary bottleneck of supervised learning. Due to the computational complexity of CO problems, collecting a sufficiently large dataset of optimal solutions is infeasible. Supervised learning cannot be effectively applied in the NCO without large, annotated datasets. Despite optimal solutions from small-scale problem instances as labels to optimize the supervised mapping, the model cannot generalize well in larger instances beyond the problem sizes seen in the training data or unseen instances due to the limited amount of training data, leading to degraded performance (Bello et al. 2016; Bengio et al. 2021; Vinyals, Fortunato, et al., 2015).

Consequently, an emerging trend in recent years is to combine NCO with reinforcement learning (RL) to overcome the drawbacks from supervised learning. This survey considers this branch of machine learning in the NCO, namely neural combinatorial optimization with reinforcement learning (NCO-RL), and its industrial engineering applications. Driven by the success of the NCO studies, the NCO-RL is deemed a promising approach to solving hard CO problems and satisfying the complex and dynamic nature of today's business requirements.

To support future research, development, and applications of NCO-RL, this survey study, as shown in Fig. 1, is essential to (i) introduce the theoretical background of RL and NCO and their combination towards NCO-RL, (ii) identify the industrial engineering applications of NCO-RL, (iii) summarize the technical limitations and missing pieces of the existing literature, and (v) suggest future research directions for NCO-RL and its applications. In order to highlight the significance and contribution of this work, the comparisons with similar survey studies describing RL, CO, and NCO-RL are summarized in Table 1.

2 Background

This section discusses the formal definitions of CO and RL, leading to the NCO-RL's algorithms, models, and terminologies as the background required to understand the remainder of the paper.

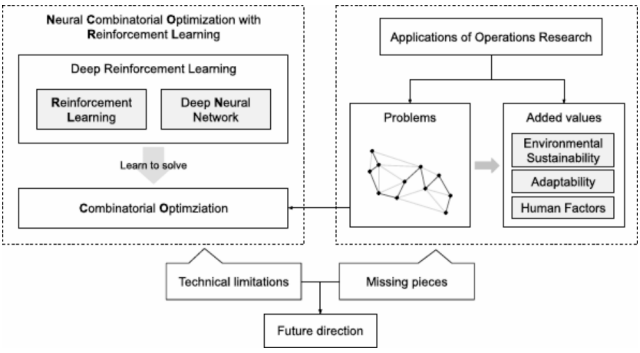


Fig. 1 An overview of this survey paper

Table 1 Comparisons between this work and related surveys

	Dimension							
References	(a)	(b)	(c)	(d)	(e)	(f)	(g)	Remarks
(Sbihi and Eglese 2010)		◯	✓		✓			Focused on green logistics
(Lin et al. 2014)		◯	✓		✓		✓	Focused on GVRP
(Li 2017)	✓		◯			✓		Focused on an exhaustive overview of RL
(Yang et al. 2020)	✓	◯	◯	◯	✓	◯	✓	Focused on energy and electric systems
(Mazyavkina et al. 2021)	✓	✓		✓		✓		Focused on classical CO problems
(Yan et al. 2022)	✓	◯	✓	◯	◯	✓	✓	Focused on decision problems
This work	✓	✓	✓	✓	✓	✓	✓	A comprehensive survey of applying RL to solve CO problems and its applications in IE.

Remarks: (a)—Background on reinforcement learning; (b)—Combinatorial optimization problems; (c)—Applications in Industrial engineering; (d)—Technical challenges of NCO-RL; (e) Added values of applications; (f) - Future directions for algorithms; (g)—Future directions for applications (✓: high coverage, ◯: medium coverage, empty: no coverage)

2.1 Combinatorial optimization

In general, a CO problem instance $P = (S, f)$ can be defined as an optimization problem with solutions decoded with discrete variables (Blum and Roli 2003), where S denotes the finite set of feasible solutions, i.e., $\{s \in S \mid s \text{ satisfies all the constraints among variables}\}$, of a given problem instance, and f denotes an objective function, $f : S \to \mathbb{R}$, to be minimized or maximized. When solving a CO problem, the goal is to find a globally optimal solution $s\prime$ of P, that is, the feasible solution with the minimum objective function value $f(s') \leq f(s)\ \forall s \in S$. S is the solution space of a CO problem, and each $s \in S$ is a candidate solution. Because S is a finite set, it is possible to enumerate and compare all candidate solutions to find the optimal value with f. However, S in hard CO problems grows exponentially with the number of variables in the problem instance.

2.2 Reinforcement learning

2.2.1 Markov decision process

Prior to the deployment of RL, the optimization problem is typically formulated as a Markov Decision Process (MDP), which provides a framework to model the sequential decision-making process in an observable system (Bellman 1957; Sutton and Barto 2018). An MDP is defined as a tuple:

$$(S, A, R, P)$$

where S is a state space, $s_t \in S$, representing the observable state at timestep t, A is an action space, $a_t \in A$, representing the action taken at timestep t, R is a reward function, $R : S \times A \to \mathbb{R}$, $r_{t+1} = R(s_t, a_t)$, mapping the immediate reward after performing an action a_t in state s_t, and P is a state transition probability, $P(s_{t+1} | s_t, a_t)$, modeling the probability of transferring from s_t to s_{t+1} when action a_t is taken in the state s_t.

Given the MDP formulation, an agent learns either a deterministic policy $\pi\ (s) = a$, mapping a state given by the environment into an action or a stochastic policy $\pi\ (a|s) = \mathbb{P}_{\pi}\ (a|s)$, mapping a state into a probability distribution of actions to maximize the cumulative reward starting from timestep t until the terminal state:

$$R_t = \sum\nolimits_{k=0}^{\infty} r_{t+k+1}$$

After an agent performs K steps until reaching the terminal state, i.e., an episode, the whole sequence of interactions between the agent and the environment is represented by a trajectory sequence $(s_t, a_t, r_{t+1}, s_{t+2}, a_{t+1}, \ldots, s_K)$. Figure 2 shows the interactions between a deterministic environment modeled as an MDP and an agent with a deterministic policy π . In the case of an unbounded horizon, an endless loop in the state transition graph of an MDP can lead to an infinite cumulative reward. To avoid this, a discounted return:

$$G_t = \sum\nolimits_{k=0}^{\infty} \gamma^{k} r_{t+k+1}$$

with a discounting factor $\gamma \in [0,1]$ is used as the function to maximize, providing a mathematical convenience to guarantee convergence (Littman and Szepesvári 1996), and hence, an MDP is solved by finding an optimal policy:

$$\pi^{*} = \underset{\pi}{\operatorname{argmax}} \mathbb{E}_{\pi}\ [G_t]$$

In stochastic domains, taking an action based on the recent state observation may not yield a maximum future reward because of its incomplete and noisy observations (Kaelbling et al. 1998). In order to allow the agent to behave effectively in a stochastic system, it is essential to use a history of observations and actions to understand the environment. A partially observable Markov decision process (POMDP) was proposed to share similar properties with an MDP without assuming the Markov property (Bengio et al. 2021; Grabski 2014). A POMDP is defined as a tuple:

$$(S, A, T, R, \Omega, O)$$

where S, A, T, and R are the typical elements in an MDP. The element Ω denotes the observation space, where $o_t \in \Omega$ represents the observation of the state received by the agent at timestep t, and O denotes the observation function $O : S \times \Omega \rightarrow [0,1]$, mapping the probability of observing o_t in s_t (Spaan 2012). The agent cannot directly observe the state information in a partially observable environment. While the policy of an MDP is a function of the present state, a POMDP agent learns a policy $\pi : H \rightarrow A$, where H

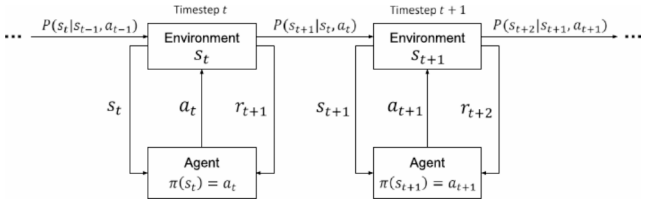


Fig. 2 Agent-environment interactions in reinforcement learning

denotes the set of observable histories, and an observable history $h_t \in H$ until timestep t is defined as $h_t = \{(a_0, o_0), (a_1, o_2), \ldots, (a_t, o_t)\}$ (Braziunas 2003).

2.2.2 Model-based and model-free RL

RL algorithms can be categorized into model-based or model-free methods. A predictive model exists for the transition function of the environment in model-based methods. After learning the model, planning algorithms, such as dynamic programming (DP) and Monte Carlo Tree Search (MCTS), are used to derive optimal actions over the model when making decisions during the training or inference stage (Li 2017; Polydoros and Nalpantidis 2017). Model-free methods do not explicitly learn and assume the availability of the underlying transition functions of the environment. Instead, model-free methods, such as Q-learning and policy gradient (PG), learn an optimal policy that is a function of the current state $\pi(s)$ without relying on the next state or next reward from a model when making decisions (Mazyavkina et al. 2021; Sutton 1988).

2.2.3 Value-based and policy-based approaches under the model-free method

Model-free methods are further classified as value-based and policy-based approaches. Most, if not all, of the reviewed studies in this survey use model-free methods. In general, model-free methods are a more popular choice because they are more time-efficient once trained. Moreover, model-free algorithms are more straightforward and flexible and apply to various tasks (Z. Zhang et al. 2020a, b). In the case of solving CO problems, fully model-based methods are less desirable because (i) the transition and reward functions are known in most cases, and thus, the learning problem is reduced to a planning problem which can be solved using DP and MCTS instead (Vázquez-Canteli and Nagy 2019), and (ii) model-free methods have a faster running time during the inference stage, which is essential especially when real-time decisions are needed in dynamic environments.

Value-based approaches Value-based methods learn a deterministic policy by considering a state-value function $V_\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$, the expected return from state s, i.e., the expectation of future discounted cumulative reward starting from state s using policy π, or an action-value function $Q_\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$, i.e., the expected return of taking action a in state s and the following policy π afterward. Alternatively, the state-value function can be written as $V_\pi(s) = \max_a Q_\pi(s, a)$ in terms of Q_π. The optimal deterministic policy $\pi^*(s) = a$ can be defined as:

$$\pi^* = \operatorname*{argmax}_\pi V_\pi(s) = \operatorname*{argmax}_\pi Q_\pi(s, a)$$

in terms of the value function. Thus, given an optimal value function, $Q^*(s, a) = Q_{\pi^*}(s, a)$ or $V^*(s) = V_{\pi^*}(s)$, an optimal policy π^* can be found to solve the MDP. However, the transition and reward function are typically rarely known, or the MDP is too large to solve (Li 2017). In such cases, it is not feasible to directly compute the optimal value functions. Instead, the value functions can iteratively converge to optimal value functions by using the well-established Bellman equation:

$$V(s) = \max_{a}(R(s,a) + \gamma V(s\prime))$$

where s' denotes the next state after taking an action a in a state s. The Bellman equation describes the relationship between the value of the current state $V(s)$ and the values of the future states $V(s\prime)$, and gives recursive decomposition based on Bellman's Principle of Optimality to allow solving MDP iteratively (Bellman 1952). The equation is applied to find the maximum expected return for each state and, thus, the optimal value or action-value function. The value functions can be expressed in the form of the Bellman equation:

$$V^*(s) = \max_{\pi} \mathbb{E}_{\pi}\left[G_t | s_t = s\right] = \max_{\pi} \mathbb{E}_{\pi}\left[r_{t+1} + \gamma V^*(s_{t+1})\right]$$

and similarly,

$$Q^*(s,a) = r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a')$$

For each timestep, the optimal action can be found by:

$$a^* = \underset{a \in A}{\operatorname{argmax}} Q^*(s,a)$$

In the case of $V^*(s)$, the optimal action can be found by:

$$a^* = \underset{a \in A}{\operatorname{argmax}} \sum_{s' \in S} P(s' \mid s,a) V^*(s')$$

$Q^*(s,a)$ is considered preferable than $V^*(s)$ because $Q^*(s,a)$ does not assume the transition function; hence, it is model-free.

Because the transition function is often unknown, value functions are approximated using model-free methods, namely the Monte Carlo (MC) method and temporal-difference (TD) learning.

- *MC method*: The value function is estimated by randomly sampling complete episodes with an initial state s and averaging the actual returns as the estimated expected return of state s (Dong et al. 2020). With many iterations, the estimated value function converges to optimality, hence an optimal policy.
- *TD learning*: A fundamental concept of TD learning is the value estimation from incomplete episodes with bootstrapping. Instead of relying on calculating the expected returns with actual rewards after a complete episode as in the MC method, TD learning estimates the immediate expected returns with a parameterized value function $V(s)$ and updates $V(s)$ towards the estimated return $V(s_t) \leftarrow V(s_t) + \alpha\left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\right]$, where α denotes the learning rate, $r_{t+1} + \gamma V(s_{t+1})$ is the TD target, and $r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ refers to the TD error. The value estimate is updated based on other value estimates, known as bootstrapping. Q-learning and SARSA are the most widely used algorithms for TD learning, which was a breakthrough in the early days of RL and is still a fundamental and prevalent method (Sutton and Barto 2018). The two algorithms are very similar in their learning procedure, but the main difference lies in

the update rules, which are further discussed in Sect. 2.2.4.

Policy-based approaches Policy-based methods aim to find a stochastic policy directly instead of value functions, effectively solving RL problems with the continuous action space (Li 2017). Because there is an infinite number of actions or states in continuous space, evaluating an optimal action $a^* = \underset{a \in A}{\operatorname{argmax}} Q^{\pi}(s,a)$ from the value function is computationally intractable. Supposed a Markov chain has a stationary state distribution $d^{\pi_\theta}(s)$, referring to the fact that an agent walks along the Markov chain forever, starting from s_0 with a differentiable policy $\pi_\theta(a|s)$ parameterized by θ, the probability of $s_t = s$ remains the same for any state, i.e., $d^{\pi_\theta}(s) = \lim_{t \to \infty} P(s_t = s | s_0, \pi_\theta)$. To find an optimal stochastic policy $\pi^*(a|s)$, the expected return function and objective function of the stochastic policy are defined as:

$$V^{\pi}(s) = \sum_{a \in A} \pi_\theta(a|s) Q^{\pi_\theta}(s,a)$$

$$J(\theta) = \sum_{s \in S} d^{\pi_\theta}(s) V^{\pi}(s)$$

By taking the derivative of the objective function with respect to θ, according to the policy gradient theorem (Sutton et al. 1999), the policy gradient in RL is defined as below for convergence:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi(a|s,\theta) Q^{\pi}(s,a)]$$

- REINFORCE (Williams 1992): Also known as the Monte-Carlo policy gradient, it updates the policy parameter θ with $\nabla_\theta J(\theta)$, where $Q^{\pi_\theta}(s,a)$ denotes the return value estimated by MC methods using rollouts, i.e., episode samples. Typically, an advantage function $A(s,a) = Q^{\pi_\theta}(s,a) - b(s)$ is used to replace $Q^{\pi_\theta}(s,a)$, subtracting a baseline value $b(s)$ from $Q^{\pi_\theta}(s,a)$ to reduce variance without changing the bias. Hence, $b(s)$, commonly as a state-value function $V_{\pi_\theta}(s)$, can be estimated using either MC methods or TD learning for policy evaluation (Li 2017).
- Actor-critic (Mnih et al. 2016) is similar to REINFORCE with baseline, both of which involve an advantage function $A(s,a)$ to reduce the variance of the policy parameter estimation. Actor-critic uses a combination of value-based and policy-based methods to reduce variance further and accelerate learning, where in addition to learning a policy function as the Actor, a value function $V^{\pi_\theta}(s_t)$ is learned for bootstrapping as the Critic to evaluate the current policy (Sutton and Barto 2018). The main difference between actor-critic and REINFORCE with baseline is within the advantage function, whereas in actor-critic methods, $A(s,a) = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$. As a result, actor-critic can be applied to MDPs with infinite horizons because it supports bootstrapping and does not rely on MC rollout estimates to evaluate the policy.

2.2.4 On-policy and off-policy learning

Apart from the value-based and policy-based classification, an RL algorithm can be implemented in either an on-policy or off-policy manner. On-policy learning uses the trajectory

generated by the target policy to train the same policy, while off-policy learning uses the trajectory generated by a different policy to train the target policy (Li 2017). Although on-policy learning provides more stable convergence properties, it can easily be trapped in local optima due to poorer exploration (Li 2017). In contrast, off-policy learning can be more sample-efficient because of the use of experience replay to store the history of trajectories and reuse them during training (Nachum et al. 2017).

To demonstrate the difference between on-policy and off-policy learning, for example, Q-learning is off-policy, and SARSA is on-policy (Sutton and Barto 2018). Both algorithms iteratively update the action-value function $Q(s,a)$ until convergence in the form of:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s', \mu(s')) - Q(s,a) \right]$$

where α denotes the learning rate, and μ denotes the policy used to find the next action a'. To encourage exploration in the early stage of training, both algorithms use ϵ-greedy strategy, i.e., the agent takes random actions with probability ϵ, which is initially set to a large value (e.g., 1.0) and gradually decreases to a small value (e.g., 0.0) over the iterations. Given that, during the training stage, the current policy π can be seen as a stochastic policy as there is a certain degree of randomness depending on ϵ. In each iteration, an action a is selected by $\pi(s)$ and $Q(s,a)$ is updated accordingly. However, the two algorithms differ in the policy μ from which the Q value of the next state $Q(s', a')$ is obtained. SARSA uses the current ϵ-greedy policy as μ, i.e., $a' = \mu(s') = \pi(s')$, while Q-learning uses a fully greedy policy such that $a' = \mu(s') = \underset{a' \in A}{\operatorname{argmax}} Q(s', a')$, as if ϵ is decayed to 0.0. Although the Q functions are the same, the policies are different. Because Q-learning uses a different behavior policy to select the next action and thus update the current target policy, it is categorized as an off-policy method.

Off-policy learning can be particularly beneficial in CO tasks because it can provide better exploration using other guided behavior policies instead of target policies in solving nonlinear and highly complex problems (Cappart et al. 2021). Details on this aspect will be further elaborated in Sect. 2.3.

2.2.5 Deep reinforcement learning

In most practical applications, when the state and action spaces are too high dimensional, complex, or even continuous, i.e., the curse of dimensionality, classical RL, such as tabular Q-learning, cannot effectively learn from the environment. Tabular Q-learning, as its name suggests, uses a table to store the Q values for all state-action pairs ($S \times A$). Such a table works well if the sizes of both state and action space are discrete and small enough, e.g., learning to navigate in four directions within a small $m \times n$ grid ($|S| = m \times n, |A| = 4$), and play Tic Tac Toe ($|S| = 3^9, |A| = 9$). However, if the discrete state space grows large, such as the $h \times w$ screen outputs as in a black-and-white Snake game ($|S| \leq 2^{h \times w}, |A| = 4$). Tabular Q-learning becomes impractical because it can take too much memory to store all the possible states and too much training time to update the table until it converges to an optimal policy. Alternatively, if either the state or action spaces are continuous, tabular Q-learning would be impractical without employing some discretization schemes.

The solution to overcome the curse of dimensionality is to use a parameterized function approximator, such as a deep neural network (DNN), to represent the functions. A feed-forward Multi-Layer Perceptron (MLP) is the simplest form of DNN. An MLP consists of an input layer, one or more hidden layers, and an output layer. The input vector $x_1, x_2, \ldots, x_n$ passes through the next layer parameterized by the weights $w \in W$ and bias $b \in B$ with an activation function ϕ. Each hidden and output layer contains multiple neurons representing the latent vector in the hidden layers and the output vector in the output layer. The activation a_j of the j-th neuron can be obtained by:

$$a_j = \phi\left(\sum {}_{i=1}^{n} w_{ji} x_i + b_j\right)$$

With a ground truth output vector $y_1, y_2, \ldots, y_m$, the model parameterized by θ learns to approximate a mapping function $\widehat{y} = f_\theta\ (x)$ by minimizing the loss between the predicted $\widehat{y}$ and the actual y values. Finding a θ that achieves zero loss (i.e., $\widehat{y} = y$) is not a trivial task. In deep learning, gradient descent is the most widely used optimization algorithm that iteratively approaches the minimum by taking small gradient steps (Dong et al. 2020). At first, θ are randomly initialized. During the training process, the weights are updated by:

$$\theta \leftarrow \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

given a differentiable loss function J, such as the mean squared error:

$$J(\theta) = \frac{1}{2n} \sum {}_{i=1}^{n} (y_i - \widehat{y}_i)^2$$

where n denotes the number of samples. For an MLP, the specific parameters are updated by $W \leftarrow W - \alpha \frac{\partial J(\theta)}{\partial W}$ and $B \leftarrow B - \alpha \frac{\partial J(\theta)}{\partial B}$.

Deep learning generally refers to neural networks with many hidden layers, such as MLPs, allowing the models to approximate complex functions (LeCun et al. 2015). In reinforcement learning, deep reinforcement learning (DRL) uses deep learning models to approximate the value and policy functions in model-free methods or the transition and reward functions in model-based methods (Li 2017). The formulation is almost the same as the Q-learning algorithm $Q(s, a)$ mentioned before, except that the Q function is parameterized by θ.

$$Q_\theta\ (s, a) \leftarrow Q_\theta\ (s, a) + \alpha\left[r + \gamma\, Q_\theta\ (s', \mu\ (s')) - Q_\theta\ (s, a)\right]$$

A notable pioneer work, Deep Q-Network (DQN), combines deep convolutional neural networks and classical RL. It has drawn enormous interest in DRL in academia and industry due to its super-human performance in playing games (Mnih et al. 2015). The introduction of deep learning models effectively resolves the previously intractable problem due to the high dimensionality in state and action spaces. Since then, more studies have attempted to apply DRL to achieve autonomous decision-making and control capabilities in complex environments (Lillicrap et al. 2015).

2.3 Neural combinatorial optimization

Current research areas of applying ML in solving CO problems can be categorized into two branches: (i) end-to-end learning and (ii) hybrid learning, as follows.

End-to-end learning trains neural networks to directly output solutions from the input. A well-known example is the Pointer Network (Vinyals, Fortunato, et al., 2015), where a recurrent encoder-decoder architecture with an attention mechanism produces a probability distribution over the input nodes for each step in TSPs and VRPs. Besides routing problems, Bello et al. (2016) solve a problem using the Pointer Network architecture to determine the subset of items packed into the knapsack. The Pointer Network is handy when the solutions of a CO problem can be represented as a permutation of the input sequence. The model addresses two significant challenges in NCO: variable-length inputs and outputs and generating a permutation of inputs, which is particularly difficult in neural machine translation tasks with general sequence-to-sequence (seq2seq) models. The model is trained with supervised learning with baseline training data generated by exact solvers. Due to the computational complexity of CO, the typical NCO embedding supervised learning is only feasible to generate optimal solutions from small problem instances.

Hybrid learning is developed to learn in conjunction with classical optimization algorithms to improve their performance in terms of computational costs and solution quality. Gasse et al. (2019) formulate the process of B&B for MILP as an MDP and introduce a GNN to learn a policy for variable selection from the strong branching (SB) rule through behavioral cloning as a classical approach for imitation learning. The learned policy thus reduces the running time, allowing more solutions to be searched within a limited period of computational duration. Similarly, Khalil et al. (2016) proposed to learn a ranking function from the SB score as the ranking to mimic the SB strategy. Alternatively, hybrid learning can serve as hyper-heuristics to select or generate a set of heuristics to solve CO problems (Burke et al. 2013).

2.4 Neural combinatorial optimization with reinforcement learning

Considering machine learning in general, it is assumed that X and Y to be the input and output space of a learnable function F, and a training set $S = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ of labeled instances sampled from an inaccessible distribution D. Optimizing F over the empirical distribution from the finite set S does not guarantee a good performance in all subsets of the unseen testing sets. An intuitive measure to reduce the generalization errors is to include more data points. In view of the weaknesses of the typical NCO discussed in Sect. 2.3, the supervised learning models in NCO cannot work well when handling large-scale instances. Moreover, Bello et al. (2016) notably show that even with optimal labeled data, a supervised agent has a poorer generalization performance than an RL agent that explores suboptimal solutions and learns from the observations. The concept of NCO-RL is illustrated in Fig. 3.

The first step of applying RL to NCO is to model the CO problem as a sequential decision process with MDP and POMDP. Several studies (Bello et al. 2016; Kool et al. 2018; Nazari et al. 2018) proposed training a seq2seq model to predict the solution of a CO problem instance directly. Given the recurrent nature of the model, the current output depends only on the initial state and previous outputs, i.e., the initial inputs and partial solution, while the

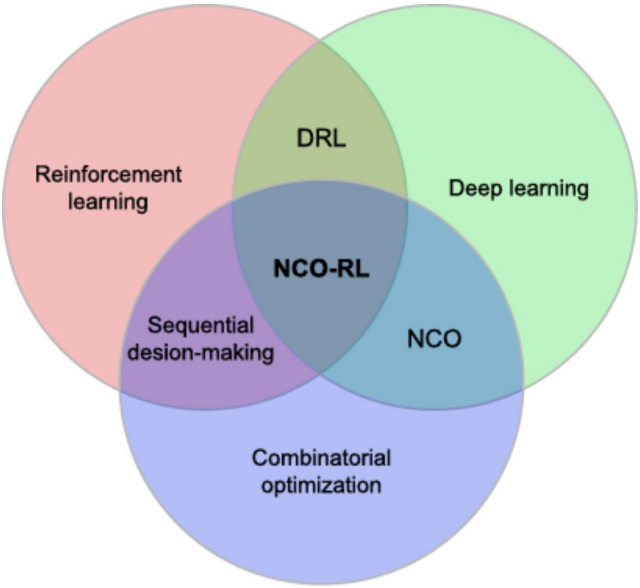


Fig. 3 The connections between deep learning, combinatorial optimization, and reinforcement learning

underlying MDP can be considered to be partially observable. As the environments of classical CO problems are primarily deterministic, a partial solution can be used as the state to determine the following action for extending the solution. Despite being formulated as an MDP, the state is essentially a trajectory of past actions, as seen in a POMDP formulation. However, the states reflecting the environment are not explicitly represented in those works (Bello et al. 2016), while some explicitly update the state embeddings based on how the partial solutions change the environment (Khalil et al. 2017), and some others incorporate both (Nazari et al. 2018). Figure 4 illustrates an overview of solving CO problems using NCO-RL, where the DQN is selected as the RL algorithm, and a replay memory buffer is used to sample trajectories as training data.

Using the NCO-RL, CO problems can be solved with deterministic or stochastic policies learned by value-based or policy-based methods. Deterministic policies learned by value-based methods are popular due to their simplicity and an inherent ability for imitation learning to train the model using hybrid learning with exact algorithms (Gasse et al. 2019). Stochastic policies are learned by policy-based methods. Sampling approaches are employed during the inference stage, which, according to Bello et al. (2016), yields better solutions than greedy selection. These approaches are instrumental in solving CO problems, where the complete model is usually known, and they can manage an acceptable computational burden when sampling solutions.

Seq2seq and struct2seq are promising choices for building the model architecture in the end-to-end approach. Using seq2seq models to learn the mapping between the inputs and outputs of a CO problem is intuitive because they are represented by variable-length sequence data (Vinyals, Fortunato, et al., 2015). Considering the permutation-invariant property in some CO problems, set2seq models, as a specialized variant of seq2seq models, are also a well-defined architecture for the NCO (Lee et al. 2019; Vinyals, Bengio, et

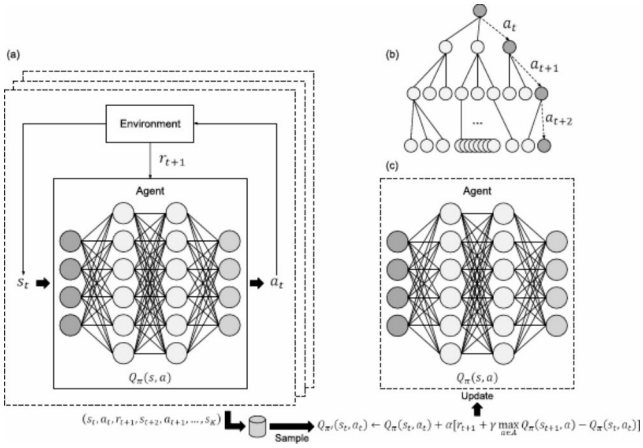


Fig. 4 An overview of solving combinatorial optimization problems with NCO-RL. **a** The agent-environment interactions during an episode give a trajectory stored in a replay memory. **b** A search-tree structure in exact algorithms, such as branch and bound, enumerates all candidate solutions over the solution space. The dash lines show the trajectory of actions in an MDP formulation, which translates to the branches that lead to a solution. **c** Given a sampled trajectory, the neural network as a Q-value function approximator of an agent is updated using the Bellman equation. An optimal policy is obtained after certain iterations of Bellman updates

al., 2015). In addition to sequences and sets, combinatorial problems can be represented through graphs in struct2seq models, which contributes similar advantages to seq2seq and set2seq models and uses GNNs to encode more information concerning the relationship of the neighboring nodes in a graph compared with sequential data (Gasse et al. 2019; Khalil et al. 2017).

3 Applications

This section reviews NCO-RL applications and summarizes their mechanisms and value in industrial engineering. A literature search on Scopus and Web-of-Science is conducted to identify relevant papers. Most papers were published in the past four years, and around two-thirds of the results are journal articles. While Scopus and Web-of-Science are the two primary sources of the search, the initial result is manually filtered, selected, and supplemented by Google Scholar if further keywords and references are found relevant to this literature search to ensure comprehensive coverage of the study.

The underlying problem types categorize the applications. The problem formulation, algorithms used, and significance of the reviewed works will be discussed. The underlying research questions covered in this section include:

- How is NCO-RL applied to solve a specific CO problem?
- Which RL methods and model architectures are available for a specific application?
- What added values are shown in the reviewed works?

There are specific vital steps and elements, as illustrated in Fig. 5, to apply NCO-RL to solve a specific CO problem. First, many real-world applications can be formulated as a CO problem of the specific problem type with the objective and constraints to derive an optimal solution from the feasible solutions. Second, given the formulation of the problem, a CO problem is modeled in MDP or POMDP. Whether the state space is implicitly learned or explicitly represented depends on the problem type and state information availability, and the action space is based on the learning scheme selected (either end-to-end or hybrid learning). Third, an RL algorithm should be selected from modeling the sequential decision process and the resulting deterministic or stochastic policy. Fourth, depending on the problem types, the general model architecture is considered for encoding the inputs, a sequence, a set, or a graph, and decoding the outputs, either a sequence or a value. Finally, given the known model of a CO problem, which is usually the case, there is an option to perform search operations, such as greedy search, beam search, and solution sampling, from the learned policy.

3.1 Vehicle routing problem

A classical capacitated VRP that contains n nodes and v vehicles can be formulated as the following MILP:

$$min \sum_{k=1}^{v} \sum_{i=1}^{n} \sum_{j=1}^{n} d_{ij} x_{ijk} \tag{1}$$

$$s.t. \sum_{i=1}^{n} x_{ij} = 1 \; \forall j \in \{1, 2, \ldots, n\} \tag{2}$$

$$\sum_{j=1}^{n} x_{ij} = 1 \quad \forall i \in \{1, 2, \ldots, n\} \tag{3}$$

$$\sum_{i=1}^{n} x_{ijk} - \sum_{i=1}^{n} x_{jik} = 0 \quad \forall j \in \{1, 2, \ldots, n\}, k \in \{1, 2, \ldots, v\} \tag{4}$$

$$\sum_{i}^{n} \sum_{j}^{n} d_{ij} x_{ijk} \leqslant Q \quad \forall k \in \{1, 2, \ldots, v\} \tag{5}$$

$$\sum_{j=2}^{n} x_{1jk} = 1 \quad \forall k \in \{1, 2, \ldots, v\} \tag{6}$$

$$x_{ijk} \in \{0, 1\} \quad \forall i, j \in \{1, 2, \ldots, n\}, k \in \{1, 2, \ldots, v\} \tag{7}$$

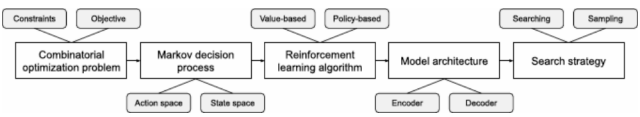


Fig. 5 Key steps to employ the NCO-RL to solve CO problems

where d_{ij} denotes the distance between node i and node j, and Q is the capacity of each vehicle. To formulate this problem as an MDP (S, A, R, P), the most intuitive components are the reward function R and action space A:

$$R(s' \mid s, a) = \begin{cases} \sum_{i=1}^{l-1} d_{i,i+1} & \text{if } s' \text{ is the terminal state} \\ 0 & \text{otherwise} \end{cases}$$

$$A = \{1\} + \{x \mid x \in \{2, \ldots, n\} \setminus \{a_1, a_2, \ldots, a_l\}\}$$

where l is the length of the trajectory of actions, and no reward is given until the terminal state. Action space A contains the depot node that can be visited multiple times and the customer nodes that have not been visited. Reward function R returns a reward equivalent to the objective value of the VRP at the terminal state. The state space S can be formulated in different ways, one of which is a concatenated route that can be expressed as:

$$s = \left(D, u_1^1, u_2^1, \ldots, u_{l_1}^1, D, u_1^2, u_2^2, \ldots, u_{l_2}^2, \ldots, D, u_1^v, u_2^v, \ldots, u_{l_v}^v, D\right)$$

$$S = \{s \mid s \text{ is a valid concatenated route}\}$$

where D denotes the depot node. l_v represents the length of the route of the v-th vehicle. $u_{l_v}^v$ is the l_v-th customer node within the route of the v-th vehicle. During the episode, the length of the state increases for each timestep. At the beginning of the episode, for example, $s = (D)$ represents the initial state. After an action a is taken, the next state $s' = (D, a)$ is modeled by a deterministic state transition function P:

$$P(s' \mid s, a) = 1$$

Hence, the state s_t is essentially the trajectory of actions at timestep t, and the terminal state represents the solution to the problem. From the above formulation, both S and A are variable-length throughout the episode. Therefore, some specific model architectures, such as recurrent networks that allow variable-length inputs and outputs, are preferred.

When comparing the MILP and MDP formulations, the flow balance constraint (4), the depot constraint (6), and the binary variable constraint (7) are inherently met by the state representation of the concatenated route. The action space applies the vehicle capacity constraint (5) by masking out the invalid actions that exceed the capacity of the last vehicle. The demand constraints (2) and (3) are met by the conditions for reaching the terminal state, i.e., the episode only terminates if all customer nodes are visited upon returning to the depot node. In addition to this classical formulation, the reviewed papers consider some specific problem variants. In the following paragraphs, the formulations of the MDPs are highlighted to understand their approaches to solving the problems.

The Pointer Network is a notable work suitable for the task proposed by Vinyals, Fortunato, et al. (2015). Bello et al. (2016) extended the Pointer Network with policy gradient algorithms to resolve the lack of optimal solutions as labeled data in supervised learning. Nazari et al. (2018) further enhanced their work with dynamic embeddings to solve a stochastic VRP (SVRP) with split delivery, where the policy is trained with A3C end-to-end. The problem is formulated as an MDP where the state is the embedding of the static ele-

ments (i.e., the node coordinates) and the dynamic elements (i.e., the current demands of the nodes and the vehicle load), which can change over time due to delivery operations, or adjustments on the customers and their demands. The action is the next node of the route, and the sparse reward is the negative total distance. The central significance of this work is its improvements over the Pointer network by replacing the recurrent encoder with embedding layers to achieve permutation invariance of the input and the inclusion of dynamic information, contributing towards a pickup-and-delivery system resilient to changing customer demands. Further work by Zhao et al. (2020) extends (Nazari et al. 2018) by replacing critic with adaptive critic and performing a local search using the model outputs as initial solutions. Similarly, Li et al. (2021) extended (Kool et al. 2018) a multi-head Transformer-based seq2seq model comparable to the work by Bello et al. (2016) with dynamic embeddings to solve the covering salesman problem (CSP).

In addition to the above seq2seq models, James et al. (2019) proposed a struct2seq variant of the pointer network trained by A3C to solve an online VRP in green logistic systems. Green logistic systems consider a fleet of autonomous electric vehicles (EVs) to have less endurance than fuel vehicles and thus require constant charging at a long driving distance from depots and renewable charging stations. While depots directly draw energy from the grid, and thus, as a virtually unlimited source of electricity, renewable charging stations have limited charging capacity. The problem is formulated as an MDP, where the state is represented by a tour graph created with a heuristic to prune the transportation network. The action for each timestep is the probability distribution of the next node to stop by. The reward corresponds to successful delivery, minimizing the total driving distance, and penalizing delayed or missed delivery and battery depletion. The system illustrated in this paper demonstrates environmental sustainability. Using EVs and renewable power stations promotes sustainability in the automotive industry with lower greenhouse gas emissions and less air pollution (Günther et al. 2015). The proposed online routing strategy performs significantly better in dynamic scenarios without extensive computation. An information center for information dispatch coordinates the vehicle fleet. The onboard requests, current battery levels, charging station outputs, and other real-time data are available to the center, which essentially builds a digital twin that can make decisions based on dynamic state information. Hence, the strategy is resilient to the new arrival and canceling of orders, and random capacities in renewable charging stations due to the unstable power supply of renewable energy sources, such as wind and solar energy.

Another approach to deal with the variable action space is to avoid operating on the search space of solutions, which translates to the action space when the problem is modeled as a sequential decision-making process but the search space of heuristics. Zhang et al. (2022a, b) proposed a hybrid Q-learning approach to solve an online container truck routing problem (CTRP) with uncertain service times. The uncertainties of the crane operation times caused by operator proficiency, weather conditions, etc., are demonstrated in the training and testing data. The problem is formulated as an MDP, where the state describes the manually defined features from the environment model, the action is the choice of heuristic, and the immediate reward is the crane idle time. The agent learns a deterministic policy as a constructive hyper-heuristic. The central significance of this work lies in the hybrid approach, which, according to the author, shows similar results in terms of the episodic reward but with considerably faster convergence because of a smaller action space and provides the interpretability of the solutions compared to the end-to-end approach. However, a

notable limitation is its generalization to other problems of the proposed method due to the handcrafted state representation. Their work achieves resilience with an online version of the classical problem.

In real-world applications, the underneath optimization problems are often more complicated. Qiu et al. (2022) applied a multiagent Hybrid PPO to solve a bi-level routing and scheduling problem of EVs in a power-transportation network. The problem is formulated as a finite POMDP with discrete time steps. EVs can buy electricity from and sell it back to the grid at different prices and locations. The POMDP defines the observation as a vector of electricity price, carbon intensity, line status, nodal load, PV generation, network location, and battery SoC, the action as a discrete routing action for potential direction selection and a continuous scheduling action for the magnitude of power charging or discharging, and the sparse reward is the weighted sum of the revenue of carbon intensity service provision, the quantity of nodal load restoration, and the energy cost penalty. The significance of this work is the design of a distributed control system of EVs, which monitors regional carbon intensity for environmental sustainability and serves as a mobile power source to restore loads of a power system in extreme events for resilience purposes.

3.2 Bin packing problem

In classical bin packing problems, the dimensions of the items are typically known, and the decision variables determine the allocations of items to bins. The following simplified formulation of a 2D bin packing problem (2DBPP) with b bins and n items does not explicitly include the constraints that prevent the overlapping of the item placements within a bin:

$$min \sum_{i=1}^{b} y_i \tag{8}$$

$$s.t. \sum_{i=1}^{b} x_{ij} = 1 \quad \forall j \in \{1, 2, \ldots, n\} \tag{9}$$

$$\sum_{j=1}^{n} a_j x_{ij} \leqslant A y_i \quad \forall i \in \{1, 2, \ldots, b\} \tag{10}$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in \{1, 2, \ldots, b\}, j \in \{1, 2, \ldots, n\} \tag{11}$$

$$y_i \in \{0, 1\} \quad \forall i \in \{1, 2, \ldots, b\} \tag{12}$$

where y_i and x_{ij} are the decision variables of the used bins and items being packed into the bin. A and a_j denote the areas of the bin and item. The objective function (8) minimizes the number of bins used. Constraint (9) guarantees that the item is packed only into a bin. The area constraint (9) ensures that the total area of the packed items does not exceed the bin's capacity if the bin is used. Constraints (11) and (12) represent the binary variable requirements.

Similar to the VRP, an MDP formulation is needed to apply NCO-RL. However, most reviewed works consider an online version of the problem where the items are unknown, except the current item will be packed. Following this, the objective changes from minimizing the number of used bins to maximizing the space utilization within a single bin. The

state and action spaces are also designed to mimic the work process of the human labors. The state space S contains the direct observation of the bin, such as the top-view pixels that visually depict the heights of packed items and the width w and height h of the incoming item:

$$D = \{(w, h) \,|\, (w, h) \text{ is the dimension of the incoming item}\}$$

$$S = \{(p, w, h) \,|\, p \text{ is the top-view pixels of the bin}, (w, h) \in D\}$$

The action space A contains the valid placement of the item, explicitly defining the item coordinate:

$$A = \{(x, y) \,|\, x \in [0, W - w], y \in [0, H - h]\}$$

where W and H denote the dimensions of the bin and ensure that the items are placed within the container's boundaries. Similar to the action space in the MDP formulation of VRPs, the invalid actions are masked out from the action space to avoid overlapping item placements. The reward function is also carefully shaped to avoid sparse rewards, i.e., there are no immediate rewards during the episode except the last step, as in the reward function in the MDP formulation of VRPs.

$$R(s, a) = \begin{cases} \frac{w \times h}{W \times H} & \text{if } s \text{ is not the terminal state} \\ 0 & \text{otherwise} \end{cases}$$

The state transition function P is no longer deterministic in this case because the dimensions of the incoming items are not known in advance. Because of this, the learned policy will not work if the distribution of the dimensions of the items suddenly changes, which implies P also changes.

$$P(s' \,|\, s, a) = \mathbb{P}(s_{t+1} = s' \,|\, s_t = s, a_t = a)$$

The difference between this MDP formulation of online 2DBPP and the MILP formulation here is quite substantial, not only because of the problem variants but also because of the more expressive representation of the state observation. It demonstrates that state space and reward function design can differ vastly from various CO problem types.

Most reviewed works apply NCO-RL to address online variants of bin packing problems. Zhao et al. (2022) applied Actor-Critic using Kronecker-Factored Trust Region (ACKTR) to solve an online 3D bin packing problem (3DBPP) for autonomous robotic packing in practical industry environments. The 3DBPP proposed in this work considers stability and order of packing to avoid collision between the packed items and the robotic arm. The MDP defines the state as a discretized height map and feasibility mask and an encoding of the dimensions of k lookahead items from the conveyor. The action is discretized and decomposes into x, y positions of the front-left-bottom (FLB) corner of an item from a top-down view, and axis-align orientations on the z-axis learned as individual actors, i.e., different actors to reduce the size of the action space with a shared critic, similar to the architecture proposed in (Tavakoli et al. 2018). The immediate reward corresponds with the volumetric

occupancy for space utilization, and a side reward is used to encourage placing items on the sides. Existing works solve offline 3DBPPs with a known list of items by, for example, the earliest work by Hu et al. (2017), which optimizes the sequence of items to be packed while the item orientation and placement decisions are made using heuristics. It minimizes the surface area so that it can adapt to real business scenarios with no fixed-sized bin. Unlike the previous works, this work considers an online version of the problem with stochastic items to adapt to unplanned orders and directly learns the placement strategy given the observations from a camera above the conveyor belt.

A more straightforward work by Kundu et al. (2019) proposed a value-based framework based on DDQN to solve an online 2DBPP in the e-commerce warehouse industry. Like (Zhao et al. 2022), this framework takes a top-down view from a camera as the input but reduces the problem to 2D by assuming an identical height of the packed items. The MDP defines the state as two discretized binary maps of the occupied grids and an incoming item, the action as the loading position, and the immediate reward as the product of the local compactness and occupied units within a cluster of connected grids. The work does not assume the knowledge of all object dimensions; therefore, it is resilient to the situation where the items arrive individually.

In addition to the above problems optimizing the space utilization of a single bin, Verma et al. (2020) applied DDQN to solve an online 3DBPP, minimizing the number of non-empty containers. The MDP defines the state as a top-down height map of the horizontal concatenation of the bins and a vector of the heights of the bordering cells surrounding the placed box, the action as the loading position, and the sparse reward as the packing fraction of non-empty bins minus the average packing fraction over all episodes to encourage continuous improvement. The DQN approximates a value function mapping a state-action pair to a Q-value instead of the more commonly seen model mapping a state to multiple action values. The decision-making is divided into two levels to reduce the size of the significant action space inherent to 3DBPPs and speed up the learning process. First, feasible combinations of location and orientation are selected using basic rules as possible actions. Second, among the possible actions, the one with the highest Q-value is chosen as the action at each time step under the current policy. Similar to the above works, this work addresses an online variant of bin packing.

Jiang et al. (2021) proposed a hybrid approach based on constraint programming and A2C to solve a 3DBPP. The novelty of this work lies in the architectural design of the actor-network, composed of a CNN sub-encoder, spare attention sub-encoder, and three attentive decoders for the sub-tasks, namely item index, orientation, and position. The problem is described as an MDP, where the state considers the box state and views the state as the input to the spare attention and CNN sub-encoder, respectively. The box state is a list of vectors describing each item's information, namely the item's packing status, dimensions, and coordinates. The view state is a discretized height map similar to (Kundu et al. 2019; Zhao et al. 2022). The action consists of selecting an item from the input sequence and the orientation and spatial position of the selected item. The immediate reward is the gap reduction between the last and the current steps to maximize space utilization, where the gap is defined as the difference between the stacked volume and the total volume of packed items. Once trained, the agent is integrated into a B&B algorithm as the CP solver by using the learned policy to decide the value for each branching variable. The work shows better scalability compared with other reviewed works, on BPPs, most of which only perform well with instances of at

most 50 items, while the author of this work claims that the proposed approach scales up to 120 items and generalizes to different numbers of items.

Puche and Lee (2022) proposed an off-policy model-based framework combining MCTS and PG, similar to (Silver et al. 2017), to solve an online 3DBPP with a buffer to allow multi-item selection at the end of the conveyor belt. The MDP formulation is the simplest one among the reviewed works on BPPs, which defines the state as the combination of a height map and dimensions for each item in the buffer, the action as the loading position of the selected item of different orientations, the immediate reward as the increase in space utilization at each time step. The output of the policy network is element-wise multiplied by an action mask to remove infeasible loading positions for all buffered items. During training and inference, MCTS is applied to explore the state space with rollout evaluation based on the learned policy, where the search tree is expanded with the feasible actions. During inference, only buffered items are observable, as opposed to the training process, where the complete item sequence is known to reduce variance and speed up training. Therefore, to maintain the assumption on stochastic item arrival, a noise is added to the training dataset so that the model can generalize to unseen item sequences.

3.3 Resource allocation problem

The remaining problem types have similar MILP and MDP formulations that vary depending on the specific description of the problems. The state and action spaces of the following applications are fixed-length and, therefore, much more straightforward to understand. The below sections highlight the MDP formulation regarding the different assumptions and descriptions of the problems stated in the reviewed papers. The focus on the choice of model architecture and RL algorithm reflects the properties of the MDP, such as the discreteness of action space, the graph representation of state space, and the stability of convergence.

Resource allocation problems (RAP) involve distributing and utilizing available resources to satisfy demand. In industrial engineering applications, RAPs are applied to various fields such as logistics, supply chain, and human management. However, depending on the case, RAPs are not inherently combinatorial unless the number of allocated resources is an integer variable.

Yu et al. (2021) applied tabular Q-learning to solve an RAP in humanitarian logistics. As a local response center, the agent learns a policy to allocate rescue resources to multiple affected areas. The MDP defines the state as a vector of deprivation state, i.e., the duration of deprivation endured for each affected area, the action as a vector of allocated resources, where the total quantity of resources at each timestep is upper-bounded, and the sparse reward as the negative weighted sum of the accessibility, deprivation, and terminal penalty cost, which measures the equity of survivors. The significance of this work in human-centricity is the inclusion of the deprivation factor as one of the metrics to evaluate the effectiveness of the allocation plans, which aims to minimize the experience of human suffering.

3.4 Job scheduling problem

The job scheduling problem (JSP) applications range from manufacturing and transportation to many other areas, in which a dynamic environment is expected if the scheduling decision must be made on the fly without access to a complete list of jobs, i.e., solving an

online scheduling problem. IoT sensors are available to collect real-time data as state features for decision-making.

Under a smart factory environment, Li et al. (2022) proposed a hybrid approach based on DDQN and the composite dispatching rules generated by genetic programming from standard heuristic rules to solve a dynamic job shop scheduling problem (JSSP) with insufficient transportation resources, i.e., AGVs. The MDP defines the state as a list of twenty-six features of jobs, operations, manufacturing resources, and transportation resources from the sensor readings, the action as the choice of a dispatching rule at each decision step, and the immediate reward as a weighted sum of makespan and energy consumption. The author considers uncertainties such as new job insertion and machine breakdowns by incorporating one or multiple disturbance events following a fixed distribution in the episodes of the training process and reflecting the dynamic change in the state features.

In practice, the physical units in a factory can be heterogeneous. Designing heuristics for good solution quality and acceptable computation time is difficult. Zhou et al. (2021) proposed a multi-agent RL framework similar to MADDPG (Lowe et al. 2017) to solve an online production scheduling problem (PSP) in smart factories with heterogeneous manufacturing units. The author considers a stochastic environment with uncertainties such as unexpected machine failure and material shortage. The MDP defines the state as a vector of tuples of operation attributes, e.g., initialization and target completion time, and heterogeneous machine states, e.g., remaining buffer time and waiting time, gathered by sensors in real-time. The action represents an operation sequence and the corresponding target machines. The reward is a weighted sum of the operation priority, waiting time, and machine utilization rate. The work handles uncertainties by online scheduling with the dynamic attributes from sensor data.

Lee et al. (2020) applied DDQN to solve a dynamic production scheduling problem in injection mold manufacturing to minimize the total weighted tardiness. The MDP defines the state as a vector consisting of the current setup type of the machine that causes additional setup time if another type of operation is scheduled, a list of fractions of waiting operations with different weights, and another list of fractions of operations with different types, for each idle machine. The action is the scheduled operation for each machine—the reward results from the operation's weight and average processing time. The proposed approach considers single-machine scheduling in that each machine is represented by an independent environment and a learning agent, with a Q-network shared across the agents. The action, i.e., the decision point, is taken whenever the machine is idle. The approach is applicable in dynamic manufacturing environments where the exact processing time of an operation is unknown at the decision point.

While machine learning-based methods have a short inference time after training, the model must be re-trained after changing the environment's parameters, as the state's size and action space are subject to change. Wang and Gombolay (2020) applied imitation learning and DQN with a size-agnostic graph attention network (GAN) to solve a multi-robot STN-based scheduling problem with temporal and spatial constraints. The MDP defines the state as the graph embeddings of the starting and finishing time of a task, partial schedules, and location information with a GAN layer, the action as the attachment of a task to the partial schedule of a robot, and the immediate reward as the change in makespan of all scheduled tasks after taking each action. Then, the state and action embeddings are passed to a Q network to estimate the expected cumulative rewards and select the optimal action.

The agent is trained with imitation learning by expert training data that is only available in a small-scale instance. It is claimed that the trained model shows the ability to generalize to more significant and unseen problems because of the size-agnostic property of both the GAN and the DQN, which maps a state-action pair to a Q-value, similar to (Verma et al. 2020), using an action embedding calculated with robot-specific features. This work considers HRC in the workspace and introduces human-centric spatial constraints on agent proximity for safe and efficient collaboration with human co-workers in a final assembly environment.

Zhang et al. (2022a, b) applied an extension of DDPG to solve a different CO problem, an assembly line balancing problem (ALBP), addressing HRC in assembly, similar to the work by Wang and Gombolay (2020). The MDP formulation defines the state as the human state S_H, robot state S_R, and execution progress state S_E, where S_H expresses the operational state (waiting, ready, in progress), the remaining time for the current task, and additionally an attenuation factor η describing human fatigue over time, and S_R is unclear but presumably similar to S_H excluding η, and S_E represents the percentage of completed tasks, the action as the assignment of tasks to both robot and human agents, the immediate rewards r_H and r_R, which show a relationship with S_E, the time spent T, and the difficulty factor D for each assembly part, and finally a sparse reward $\sum_{S_E=0}^{S_E=1} T + \mathbb{E}(D_H, D_H^t, D_H^m)$ where D_H^t and D_H^m denote the difficulty factors for additional tools and parts, respectively, required to perform the task. The extension of DDPG implements two agents individually with actor and critic networks for humans and robots to learn different policies with different state representations and reward functions while in the same environment. The TD errors from the critic networks directly update the agents, and the shared TD error calculated from a weighted sum of the estimated Q values indirectly updates both agents to add global information. Given the above, the author highlights human centricity by adopting the human fatigue factor η in the reward calculation.

Wang et al. (2021) presented an end-to-end method with a PPO algorithm to solve the dynamic job shop scheduling problem (JSSP) in smart manufacturing. The dynamic JSSP is modeled as an MDP, where the state is defined as three matrices of job processing status, designated machine, and processing time, the action is a vector of operations assigned to each idle machine, and the reward function is defined as the difference in machine utilization $U_t - U_{t-1}$, where U_t is the ratio of the total processing time of all scheduled jobs at time t to the current completion time, after taking the action, and adds an extra reward signal of $\frac{100}{\max(makespan-L,1)}$ at the terminal state to guide the learned policy towards a theoretical optimal makespan L. The method deals with uncertainties, such as unexpected changes in processing time and machine sequence, to ensure resilience through online learning.

In addition to the above applications on manufacturing, Zhang et al. (2020a, b) applied DQN to solve an EV charging scheduling problem (EVCSP) to minimize the total charging time of EVs with a maximal reduction in the origin-destination distance. The problem is formulated as an MDP. The state consists of the number of available fast and slow charging stations. The action defines the scheduled EV, the charging station, and a binary variable representing fast or slow charging. The immediate reward corresponds to the finish charging time of the current EV and the percentage of origin-destination (OD) distance reduction, i.e., the difference between the initial OD distance and the resulting OD distance after taking the action. The significance is two-fold: First, the application promotes the effective charging of EVs as an alternative means of transportation to reduce carbon emissions and fossil fuel

consumption, hence contributing to environmental sustainability. Second, the OD distance reduction objective is human-centric, such that the scheduling strategy considers EV users' personalized destinations. Wan et al. (2018) applied DQN to solve a discretized EVCSP, optimizing charging and battery degradation costs due to charging cycles.

3.5 Task assignment problem

Most of the reviewed work involving TAP is applied to mobile crowdsourcing or edge computing services. Sun and Tan (2019) applied a variant of dueling DQN (Wang et al. 2016) to solve an online trust-aware task allocation problem (TTAP) with uncertainties in mobile crowdsourcing applications. The MDP defines the state as the spatial task represented by the location coordinates, completion deadline, and task type. The action allocates a mobile worker to a spatial task. The immediate reward is the weighted sum of the normalized values of the quality attributes, namely the trust score of the mobile worker and the travel distance between the mobile worker and the spatial task. Task allocation is essential to mobile crowdsourcing platforms, such as Uber and Didi Chuxing, which embrace a human-centric business model that benefits both companies and workers, and the policy of the RL agent is resilient to the uncertainties of the environment where all parameters can change spontaneously. A similar work by Shi et al. (2019) used a multi-agent DQN-based framework to solve an online EV fleet dispatch problem (EVFDP) for ride-hailing services to minimize the total customer waiting time, electricity consumption, and vehicle operational costs.

Motivated by (Zhang et al. 2018), Lu et al. (2020) presented an asynchronous federated learning (AFL) scheme with blockchain and RL to solve a dynamic TAP, addressing the security, reliability, and efficiency of edge device data sharing in the Internet of Vehicles (IoV). The policy is trained using the DDPG algorithm. The author proposed a use case for AFL in vehicular networks where the vehicles are heterogeneous in terms of computing capacity and dynamic communication condition, and hence, the running time of an iteration depends on the slowest vehicle node. The role of RL lies in learning a policy to perform node selection for AFL. The MDP formulation defines the state as a tuple of the dynamic wireless data rates, available computing resources, locations, and selection states of the vehicles, the action as a binary vector of selected nodes, and the reward as the average sum of the time cost and the quality of learning, describing the accuracy of the local model for each node, of the selected vehicles. The author also proposed a hybrid blockchain, PermiDAG, to achieve reliable and secure data transmission for the model parameters of AFL. The interested reader should refer to this paper for further details. While blockchain enhances cyber security, RL increases network resilience towards the dynamic environments in the IoV.

In addition to the above end-to-end approaches, Tao et al. (2022) applied a hybrid approach based on DQN and the Kuhn-Munkres algorithm to solve an order dispatch problem (ODP) with mobile power sources similar to the work by Qiu et al. (2022). The MDP defines the state as a vector of discrete periods, level of energy storage, and coordinates. The action chooses to serve a particular order, return to the depot for battery swapping, or stay idle. The immediate reward defines the revenue of the order and the fuel cost with a significant penalty in case of insufficient energy storage for serving the order. The DQN model learns from historical and simulation data in an offline manner. The learned state-action value is used as the weight of each edge in the bipartite graph for the online order dispatch

with maximum weight matching. The resulting policy shows advantages in enhancing the system resilience, namely the load restoration rate, in case of contingencies in the electricity grids.

3.6 Energy supply planning

Dehghani et al. (2021) proposed an end-to-end framework based on A2C to solve a combinatorial planning problem with uncertainties in power distribution systems to enhance lifecycle resilience via adequate pole replacement. The RL environment models the uncertainties, including the occurrence and intensities of a stochastic series of hurricanes, hazard-induced damage, and the recovery processes. A power distribution system consists of many utility poles divided into N_C clusters of equal sizes based on their risk-based ranking index recomputed every timestep. The system's topology is assumed to be static over time, and the maximum number of replacements is constrained. The sequential decision-making problem is formulated as an MDP, where the state is a vector of size N_C that describes the average time to the latest pole replacement for each cluster. The action is a binary vector of size N_C which consists of two actions, replacing all or none of the poles, for each cluster, and two alternative reward functions, namely, the Expected Outages, which calculates the probability of failure of the pole, and the expected resilience, which considers the number of customers without power, are proposed. The significance of the framework can be demonstrated in resilience by considering multiple stochastic events challenged by the inherent uncertainties in operation to ensure its reliability and robustness under extreme weather events.

Another work on energy management by Munir et al. (2019) applied DQN to solve an energy supply planning problem (ESPP) in a multiaccess edge computing (MEC) environment with a microgrid. The problem is formulated as an MDP. The state consists of the energy demand, renewable and non-renewable energy generation, and stored energy in the microgrid. The action determines the energy storage and buying decision from the main grid, and the reward corresponds with the total energy consumption under the energy supply plan derived from the policy. The uncertainties come from the random energy demand and generation rate.

4 Discussion

In this section, we first summarize the added values of the applications in terms of (i) environmental sustainability, (ii) adaptability, and (iii) human factors to identify the potential research gaps. Moreover, the technical limitations of applying the NCO-RL in solving real-world problems are discussed, while the corresponding future research directions are suggested.

4.1 Added values from the NCO-RL

To highlight the real-world significance and added values of the reviewed applications, the methods, and modeling of the problems are compared against previous studies.

Environmental sustainability is primarily demonstrated in the application domain of optimization problems. There is a common focus among the reviewed papers, namely, James et

al. (2019) and Wan et al. (2018), on intelligent transportation systems with electric vehicles and power systems. This application domain introduces several challenges when compared to conventional transportation systems. One significant issue is the additional complexity in the decision variables and constraints addressing the fluctuating electricity price and charging process. Another gap in existing research is the application of last-mile delivery. It is hard to achieve home delivery planning because of the lack of support for online changes and the slow system response time using conventional approaches. NCO-RL approaches contribute to solving time-sensitive CO problems more effectively and can be further investigated in these emerging applications.

Adaptability is one of the main advantages of solving CO problems using NCO-RL over conventional approaches. The difference among these methods that lead to better adaptability is how the problem is formulated. The existing studies are typically based on MILP formulation, which requires enormous computation time to solve by mathematical programming methods. If a variable that reflects the real world is modified, the entire solution procedure needs to be rerun to obtain an updated solution, which is not practical in scenarios that require responses in a timely manner. Unlike MILP, NCO-RL applications are based on MDP formulation, a sequential decision-making framework that allows stochastic transitions reflecting uncertainty in real-world systems. Notably, Qiu et al. (2022) and Tao et al. (2022) consider multi-agent scenarios where agents must learn cooperative and competitive strategies that lead to high rewards. Such policies are highly complex and challenging to solve using conventional methods. Most reviewed papers take advantage of this property, making adaptability the most value-added aspect.

Human factors are highlighted in the application scenarios, specifically human-robot collaboration in manufacturing environments (Wang and Gombolay 2020; Zhang et al. 2022a, b). Humans and robots are typically responsible for different subtasks within a collaborative task because of their respective strengths and limitations. In real-world applications, the inherently uncertain nature of human behavior cannot be overlooked. The reviewed applications propose simulation environments where human and robot agents operate collaboratively. The heterogeneous agents learn different policies, in which human policies are typically more noisy, and robot policies need to adjust to this variability. More applications of NCO-RL can be developed in this emerging research area towards Industry 5.0.

Among the reviewed works on NCO-RL applications, as shown in Table 2, most show adaptation to dynamic environments and problem instances of different sizes. However, applications considering environmental sustainability are relatively lacking. Those with such considerations are primarily in energy sectors or involve EVs or mobile crowdsourcing in the application domain. Some reviewed works involve human factors to ensure workers' or users' safety, needs, and benefits.

Furthermore, the performance of the reviewed works is evaluated using the percentage improvement or gap in specific metrics over other methods. If available, information regarding the sizes of the problem instances and computation time is also provided. The performance in model convergence is reported in the absence of performance comparison.

4.2 Technical challenges

Applying neural networks and RL to solve CO problems is not trivial. Several challenges and limitations arise from the nature of CO. Although the use and combination of neural

Table 2 Summary of value-added applications of the NCO-RL

Reference	Problem	Function approximator	Method	(a)	(b)	(c)	Performance evaluation
(Nazari et al. 2018)	SVRP	Pointer Network + embedding encoder	A3C		✓		10 and 13% gap in total vehicle distance over a MILP solver (n = 10, 20)
(Zhao et al. 2020)	VRPTW	Pointer Network + embedding encoder	Adaptive actor-critic		✓		1.02%, 2.48%, and 4.37% gap in total vehicle distance over Gurobi and 70–80% reduction in computation time (n = 20, 50, 100)
(Li et al. 2021)	CSP	Transformer	REINFORCE		✓		1.40%, 1.44%, and 0.84% gap over a local search method and 95–99% reduction in computation time (n = 150, 200, 300)
(James et al. 2019)	Online VRP	Pointer Network + struct2vec	A3C	✓	✓		33.66% and 39.90% improvement in the request served and distance over a MILP solver using the same computation time
(Y. Zhang et al. 2022a, b)	Online CTRP	Q-network + constructive heuristics	DDQN		✓		7.39% and 8.20% improvement over manual heuristic
(Qiu et al. 2022)	Bi-level Routing and Scheduling Problem	Policy network	Multiagent PPO	✓	✓		195% improvement over a rule-based method
(Zhao et al. 2022)	Online 3DBPP	Policy networks	ACKTR		✓		22.30% improvement over a heuristic rule using 178% more computation time
(Hu et al. 2017)	3DBPP	Pointer Network + heuristics	REINFORCE		✓		5% improvement over a heuristic method (n = 8, 10, 12)
(Kundu et al. 2019)	Online 2DBPP	Q-network	DDQN		✓		7–26% improvement over heuristic methods
(Verma et al. 2020)	Online 3DBPP	Q-network	DDQN		✓		8.51 − 18.35% improvement over heuristic methods using 223% more computation time
(Jiang et al. 2021)	3DBPP	Attentive network + B&B	A2C		✓		3.5%, 9.3%, 15.4% and 22% improvement in space utilization over a genetic algorithm (n = 20, 30, 50, 100)
(Puche and Lee 2022)	Online 3DBPP	Policy network + MCTS	PG + MCTS		✓		23.6%, 8.71, −1.2% improvement in space utilization over heuristics
(Yu et al. 2021)	RAP	Q-table	Q-learning			✓	0% optimality gap over dynamic programming and 31.74% reduction in computation time for the large instance and 459.68% more computation time

Table 2 (continued)

Reference	Problem	Function approximator	Method	(a)	(b)	(c)	Performance evaluation
(Li et al. 2022)	Dynamic JSSP	Q-network + constructive heuristics	DDQN		✓		12.45% improvement in objective value over baseline Q-learning
(Zhou et al. 2021)	Online PSP	Policy network	MADDPG		✓		20.5% improvement in makespan over a genetic algorithm
(Lee et al. 2020)	PSP	Q-network	DQN		✓		59.49–79.51%, 39.21–52.87%, 38.61–49.55% and 38.91–47.52% improvement in total weighted tardiness over heuristics ($n = 10$, 30, 50, 70)
(Wang and Gombolay 2020)	STN-based JSP	Q-network + GAN	DQN + imitation learning		✓	✓	−4.38%, −1.88%, and 10% improvement in makespan over Gurobi solver ($n = 16$–20 (small), 40–50 (medium), 80–100 (large))
(Zhang et al. 2022a, b)	ALBP	Policy network	DDPG			✓	Achieve 7s total waiting time with a total assembly time of 243s
(Wang et al. 2021)	Dynamic JSSP	Policy network	PPO		✓		0–12.87% optimality gap in makespan
(Zhang et al. 2020a, b)	EVCSP	Q-network	DQN	✓		✓	42.64% and 60.45% improvement in average charging time over heuristics
(Wan et al. 2018)	EVCSP	Q-network	DQN	✓			2.26%, 7.43%, 18.2% and 31.01% improvement in cost reduction over model predictive control (MPC) + LSTM, MPC + NN, Day-ahead, and fitted-Q iteration (FQI)
(Sun and Tan 2019)	Online TTAP	Q-network	Dueling DQN		✓	✓	Optimal policy reached after around 15,000 iterations.
(Shi et al. 2019)	Online EVFDP	Q-network	Multiagent DQN		✓	✓	3.20% and 18.12% improvement in cost saving over an optimization-based method and greedy algorithm
(Lu et al. 2020)	Dynamic TSP	Policy network	DDPG + AFL		✓		125% improvement in global accuracy of local model over local CNN
(Tao et al. 2022)	ODP	Q-network + Kuhn-Munkres algorithm	DQN		✓		73.68%, 15.00%, and 23.99% improvement in average cost saving, finish rate, and distance over nearest rule heuristic

Table 2 (continued)

Reference	Problem	Function approximator	Method	(a)	(b)	(c)	Performance evaluation
(Dehghani et al. 2021)	Planning problem	Policy network	A2C		✓		29.8% and 30.2% improvement in cumulative expected resilience over a heuristic method
(Munir et al. 2019)	ESPP	Q-network	DQN		✓		5.7% and 7.3% improvement in energy saving over random greedy and greedy methods

Remark: (a) Environmental sustainability, (b) Adaptability, (c) Human factors. In the column of function approximator, Q-networks and Policy networks refer to deep neural networks with various architectures composed of CNNs and MLPs

networks and RL address problems, such as the expensive computational cost and lack of training data compared with other existing approaches, several major architectural and algorithmic issues remain challenging. The following subsections present a list of issues concerning the underlying CO problems.

4.2.1 Insufficient exploration in large discrete action space

Solving CO problems with RL requires formulating an MDP with a discrete action space. For allocation and scheduling problems, the action space is typically small and of low dimensionality. However, the action space size can be large and proportional to the size of the problem instance when applied to solve routing problems (Nazari et al. 2018; Zhao et al. 2020), but this concerns the algorithms' scalability. Some problems inherently have a large action space, e.g., the action space of 3DBPPs is a Cartesian product of item selection, loading position, and orientation (Jiang et al. 2021; Puche and Lee 2022). The large discrete action space poses a challenge to both the capacity of the neural networks and the exploration of the RL agent. This limitation can be extended to address the robustness of the models and the potential risks in real-world scenarios if a model is deployed. If the model lacks exploration of the possible states of the environment, it may not generalize well on those unseen cases. If such a model is applied in unpredictable real-world situations, it can make decisions that lead to risky situations without warning. Because of that, careful analysis and validations should be performed to address the impact from environmental variability. Continual learning is also an essential step to achieve model robustness.

4.2.2 Slow convergence with sparse reward

CO problems are defined based on an objective function with constraints. While the exact objective function is known, naïvely applying it as a sparse reward, i.e., receiving the objective value as a reward at the terminal state and no reward at intermediate states, to the agent can lead to data inefficiency, slow learning, and even difficulty in convergence. While the return G_t resolves the sparsity to a certain degree where the discount factor γ assumes the contribution of each action solely based on the time interval between states, it is not trivial to identify which action or actions cause the outcome given a long trajectory, commonly known as temporal credit assignment (Sutton 1984). Depending on the problem types, the

objective value can be causally shaped into immediate rewards, e.g., using the increase in space utilization of a BPP as an immediate reward r_t at each time step (Jiang et al. 2021; Kundu et al. 2019; Puche and Lee 2022; Zhao et al. 2022) and hence, by summing the increases, the final space utilization is obtained as the cumulative reward R_t. Otherwise, further research is needed to adopt different RL methods and inject domain knowledge to shape rewards and guide learning.

4.2.3 Ineffective data representation

Unlike game-playing agents using raw pixels from the screen as state representation, the size of the state space changes during an episode in some of the reviewed works. For example, (Wang and Gombolay 2020) considers the partial solution as part of the state representation, and a size-agnostic GAN is applied to encode state observations to a state embedding. While many CO problems can be represented as a graph, it can be tricky to apply suitable model architectures to incorporate graphs into RL models (James et al. 2019). Alternatively, (Zhang et al. 2022a, b) use domain knowledge to explicitly design the state representation to describe an overview of the environment instead of features of individual nodes (Li et al. 2022; Wang et al. 2021), but it is challenging and requires further modifications of the MDP. On the other hand, as a strategy to address the issue of feasibility or generalization, (Verma et al. 2020; Wang and Gombolay 2020) learn value functions mapping state-action pairs to Q-values, which involves learning an action representation before or during training. Hence, further work is needed to investigate and propose state-action representations, and more experiments should be conducted to determine the best configuration for each application.

4.2.4 Poor generalization to different problem sizes

Most of the reviewed works focus on problem instances of small size, e.g., VRPs with up to 100 nodes (James et al. 2019; Nazari et al. 2018; Zhao et al. 2020), single-bin BPPs with 50 items (Jiang et al. 2021; Zhao et al. 2022), etc., where existing exact algorithms work on larger instances, e.g., VRPs with 200 nodes (Toth and Vigo 2002) and 3DBPPs with 90 items (Martello et al. 2000). Further research is needed to evaluate and improve performance on larger problem instances. In addition, because NCO-RL is a learning-based approach, once trained, evaluating problems of unseen sizes can be a limitation, and it is not practical to train a new model from scratch for each problem size. Therefore, the model must generalize to unseen sizes, as the problem size is mostly not fixed in real-world applications, such as VRPs.

4.2.5 Difficulties in finding feasible solutions

CO problems come with constraints to ensure a feasible solution, and sometimes, with more challenging constraints, the feasible action space is only a small subset of the action space in terms of MDP formulation, e.g., only a few actions are feasible in 3DBPPs due to the stability and positioning constraints. Finding feasible solutions to NP-hard problems is complex and even more challenging in machine learning (Bengio et al. 2021). In reinforcement learning, because a CO problem is formulated as a sequential decision process, the feasibility of solutions is addressed by the validity of actions. For example, end-to-end agents can

learn to take valid actions by action masking during both training and inference (Puche and Lee 2022). However, enforcing constraints, or in other words, masking invalid actions prematurely during training, can lead to early termination and large unexplored action space, hence increasing difficulty in learning. Apart from this, generating action masks can be computationally intensive as they scale with the size of the action space. Some possible ways to alleviate these problems are, e.g., to soft-constrain the action space with a penalty in the early stage of training to allow exploration (Farazi et al. 2022), by curriculum learning (Bengio et al. 2009), different schemes to generate and apply action masks (Huang and Ontañón 2020).

4.2.6 Uncertain optimality gaps

Despite the short running time, the optimality gap of the NCO-RL solutions still needs further improvement. While some are reportedly almost equal to the optimal ones, they can only be claimed on problem instances of small size and less complex CO problems. The reasons are multi-fold, e.g., the capacity of the neural network as a value or policy function approximator, poor exploration, fluctuating and unstable convergence, the lack of searching for better trajectories during inference, and many more. Further research should be done on other architectures, algorithms, and learning schemes apart from the existing works to improve solution quality.

4.2.7 Computational inefficiency in complex models

NCO-RL relies on deep learning models to optimize the near-optimal policy. The performance of these models, in terms of both optimality and computational efficiency, is often evaluated by comparing them with other algorithms as the baseline to solve the same problem. A general rule of thumb is that the more complex the model, the more optimal the solution quality because a complex model can map more intricate relationships between the inputs and outputs. However, complex models such as transformers take longer training time to converge to optimal policy and have higher inference times. As a result, efficient model inference, which demands less computational power while minimizing the impact on model performance, has become an important research direction.

4.3 Future directions

This section further suggests promising future research directions, addressing the earlier technical challenges and exploring interdisciplinary applications in emerging sectors.

4.3.1 Non-stationary distributions in stochastic environments

Most reviewed studies consider a dynamic environment with online decision-making, where a stationary distribution shows stochasticity in the environment. However, many real-world applications are characterized by non-stationary distributions. In general, non-stationary distributions are described as (i) a fixed distribution function with varying parameters, e.g., a steadily increasing mean for the demand over time, or (ii) different distribution functions with varying parameters in case of an abrupt change in the variables. If an underlying non-

stationary distribution is anticipated, it is possible to utilize change-point detection algorithms to capture statistical changes. This approach enables the selection of a policy that is trained under the same distribution assumption. Alternatively, temporal information can be encoded as state variables, allowing the model to learn the patterns. Otherwise, with an online learning scheme, the agent learns continuously from new problem instances to adapt to the change in distribution. Special considerations should be given to avoid catastrophic forgetting (Kirkpatrick et al. 2017).

4.3.2 Hybrid learning with different heuristics

While the majority of the reviewed studies employ end-to-end learning, hybrid learning with heuristics can be beneficial in reducing the action space, dealing with variable action space, generalizing to unseen problem sizes because of the size-agnostic pool of heuristics, and speeding up training due to the lesser need to explore a large discrete action space. However, domain knowledge is necessary to select heuristics for different CO problems. It is possible to automatically generate heuristics by genetic programming (Burke et al. 2009) to reduce the dependency on domain knowledge or design more sophisticated functions to improve solution quality. In addition to constructive heuristics, learning with local search heuristics or combining the two is an alternative, but it can be challenging to design appropriate rewards and termination conditions.

4.3.3 Bi-level combinatorial optimization with hierarchical reinforcement learning

Many complex real-world applications can be understood as bi-level CO problems. For example, considering e-commerce delivery services, the aim is to determine vehicle routes with feasible three-dimensional loading plans, formulated as a bi-level problem combining a VRP and a 3DBPP. Existing studies are limited because of the high combinatorial complexity of the problem (Fuellerer et al. 2010). NCO offers a computationally tractable and promising alternative to solve hard bi-level CO. Hierarchical reinforcement learning can naturally fit a bi-level optimization problem by decomposing it into sub-problems to handle the upper- and lower-level optimization tasks independently.

4.3.4 Model-based searching and sampling

In the case of CO problems, the model of the environment is usually known, even with stochasticity, and thus, unless the aim is to explicitly learn the stochastic dynamics of an MDP, dynamics model learning is not particularly well-suited. Therefore, most of the reviewed studies employ model-free methods. A few studies combine model-free and model-based methods in the planning algorithm to exploit the learned value function to guide the search given a known model. Alternatively, candidate solutions can be sampled using the learned policy function, and subsequently, the known transition and reward functions are used to check the feasibility and evaluate the quality of the solutions, respectively. These strategies can be applied during inference only or both training and inference. Although searching and sampling strategies typically carry a heavy computational burden, running simulations under a CO environment is typically computationally inexpensive, making them helpful in

improving optimality. Further research can be done to explore the effectiveness of different search and sampling schemes.

4.3.5 Size-agnostic state-action encoder

The state and action representations of NCO-RL can vary in size throughout the episode. To address this issue, size-agnostic state-action encoders are employed to embed the state-action information into fixed-sized representations to feed into the neural networks. It is straightforward to utilize recurrent networks such as RNNs and LSTMs to, e.g., encode input sequences or partial solutions, but they can be challenging to train and do not perform well on long sequences (Li et al. 2019). GNNs may be ideal because most CO problems can be represented as a graph (Cappart et al. 2021). Transformers and attentive models are also desirable for being powerful learners, as shown by recent advances (Chen et al. 2021; Vaswani et al. 2017).

4.3.6 Imitation learning and curriculum learning

To alleviate slow convergence resulting from large discrete action space and sparse reward, behavioral cloning, an approach for imitation learning, can be applied to learn an initial policy from demonstrations, i.e., optimal solutions using exact algorithms or sub-optimal solutions using heuristic algorithms in case of large problem instances. If the solution optimality and generalization to unseen instances are not satisfactory, further training can be done based on the typical exploration strategies of the selected RL algorithm. Alternatively, local search can be applied to find neighboring solutions from the solutions obtained by the learned policy to train the current policy further, which can be seen as a form of implicit curriculum learning and does not preclude the discovery of better policies from the provided demonstrations. However, such strategies only apply to end-to-end approaches and may require special attention in stochastic environments.

4.3.7 Model compression

Model compression techniques such as network pruning, weights quantization, and knowledge distillation are widely used to reduce the network size and computational complexity of deep learning models (Chung et al. 2024; Pei et al. 2023). However, most applications of model compression techniques are on supervised learning tasks. The primary difference between supervised learning and RL tasks is the learning target. In supervised learning, the model is trained on labeled data where the expected output target is known. In contrast, reinforcement learning tasks iteratively improve either the value or policy function when interacting with the environment. Instead of receiving direct target outputs to minimize the loss, the target keeps changing until the policy converges to a stable state. This dynamic nature challenges the existing model compression techniques, as RL tasks often lack a complete and static ground truth. Recent research has addressed this by developing training schemes that incorporate model compression into the update steps of RL, enabling the deployment of RL agents on resource-constrained systems (Su et al. 2024; Xie et al. 2021).

4.3.8 Interdisciplinary applications in emerging sectors

NCO-RL presents unique advantages in emerging sectors with increasing complexity, uncertainty, and dynamic requirements compared to traditional industries. These sectors demand real-time decision-making, adaptability to changing environments, and computational efficiency, making NCO-RL an ideal choice for the solution framework. To name a few emerging sectors, NCO-RL can optimize several operations such as electric vehicles routing and scheduling; the allocation of electrical grids for green logistics, demand forecasting of energy consumption. Similarly, in smart cities, NCO-RL can be applied to address challenges such as urban mobility and resource allocation to ensure fairness and reliability. These research areas come with high uncertainty and dynamic variables, such as the fluctuation of energy demand and supply tougher with dynamic behavioral patterns. NCO-RL can contribute significantly by utilizing real-time conditions and making timely decisions in these domains.

5 Conclusion

This survey introduces the theoretical background of CO, NCO, RL, and their combination, NCO-RL, and reviews the applications of NCO-RL in industrial engineering problems. We categorize the applications based on the types of CO problems and describe the MDP formulations, architectures, and algorithms to solve the problems while highlighting the added values. The missing pieces of the applications and the technical challenges and limitations are summarized, and finally, some future directions are suggested.

To conclude, although the works reviewed in this paper are still in the development stage compared with existing algorithms, we see the emerging trend and recent progress and believe that NCO-RL could be a promising direction for solving combinatorial optimization problems. Regarding its application in industrial engineering, NCO-RL can be a valuable approach to improving decision-making, especially when real-time decisions are needed in a dynamic environment. Moreover, by considering environmental sustainability, adaptability, and human factors in policy behavior, NCO-RL is a general and versatile solution to support operations processes in diverse and rapidly changing industries and adapt to the transition from Industry 4.0 to Industry 5.0.

Acknowledgements The authors would like to thank the Research and Innovation Office of The Hong Kong Polytechnic University for supporting the project. (Project code: RK0W)

Author contributions K.T Chung wrote the manuscript, CKM Lee secured the research grant and edited the manuscript. YP Tsang revised the manuscript.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it.

The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Bellman R (1952) On the theory of dynamic programming. Proc Natl Acad Sci 38(8):716–719
Bellman R (1957) A markovian decision process. J Math Mech, 679–684
Bello I, Pham H, Le QV, Norouzi M, Bengio S (2016) Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*
Bengio Y, Louradour J, Collobert R, Weston J (2009) Curriculum learning. Proceedings of the 26th annual international conference on machine learning
Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: a methodological tour d'horizon. Eur J Oper Res 290(2):405–421
Blum C, Roli A (2003) Metaheuristics in combinatorial optimization: overview and conceptual comparison. ACM Comput Surv (CSUR) 35(3):268–308
Braziunas D (2003) POMDP solution methods. University of Toronto, Toronto
Burke EK, Hyde MR, Kendall G, Ochoa G, Ozcan E, Woodward JR (2009) Exploring hyper-heuristic methodologies with genetic programming. Computational intelligence. Springer, Cham, pp 177–201
Burke EK, Gendreau M, Hyde M, Kendall G, Ochoa G, Özcan E, Qu R (2013) Hyper-heuristics: a survey of the state of the art. J Oper Res Soc 64(12):1695–1724
Cappart Q, Chételat D, Khalil E, Lodi A, Morris C, Veličković P (2021) Combinatorial optimization and reasoning with graph neural networks. In: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence
Carrera DG, Mack A (2010) Sustainability assessment of energy technologies via social indicators: results of a survey among European energy experts. Energy Policy 38(2):1030–1039
Chen L, Lu K, Rajeswaran A, Lee K, Grover A, Laskin M, Abbeel P, Srinivas A, Mordatch I (2021) Decision transformer: reinforcement learning via sequence modeling. Adv Neural Inf Process Syst 34:15084–15097
Chung K, Lee C, Tsang Y, Wu C, Asadipour A (2024) Multi-objective evolutionary architectural pruning of deep convolutional neural networks with weights inheritance. Inf Sci 685:121265
Clausen J (1999) Branch and bound algorithms-principles and examples. Department Comput Sci Univ Cph, 1–30
Colabianchi S, Costantino F, Di Gravio G, Nonino F, Patriarca R (2021) Discussing resilience in the context of cyber physical systems. Comput Ind Eng 160:107534
Dehghani NL, Jeddi AB, Shafieezadeh A (2021) Intelligent hurricane resilience enhancement of power distribution systems via deep reinforcement learning. Appl Energy 285:116355
Dekker R, Bloemhof J, Mallidis I (2012) Operations Research for green logistics–An overview of aspects, issues, contributions and challenges. Eur J Oper Res 219(3):671–679
Dong H, Dong H, Ding Z, Zhang S (2020) Deep reinforcement learning. Springer, Cham
Farazi NP, Zou B, Tulabandhula T (2022) Dynamic On-Demand crowdshipping using constrained and Heuristics-embedded double dueling deep Q-Network. Transp Res E 166:102890
Fuellerer G, Doerner KF, Hartl RF, Iori M (2010) Metaheuristics for vehicle routing problems with three-dimensional loading constraints. Eur J Oper Res 201(3):751–759
Gasse M, Chételat D, Ferroni N, Charlin L, Lodi A (2019) Exact combinatorial optimization with graph convolutional neural networks. Adv Neural Inform Proc Syst. https://doi.org/10.48550/arXiv.1906.01629
Grabski F (2014) Semi-Markov processes: application in system reliability and maintenance. J Polish Saf Reliab Assoc, 5
Günther H-O, Kannegiesser M, Autenrieb N (2015) The role of electric vehicles for supply chain sustainability in the automotive industry. J Clean Prod 90:220–233
Hu H, Zhang X, Yan X, Wang L, Xu Y (2017) Solving a new 3d bin packing problem with deep reinforcement learning method. *arXiv preprint arXiv:1708.05930*
Huang S, Ontañón S (2020) A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint arXiv:2006.14171*
James J, Yu W, Gu J (2019) Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. IEEE Trans Intell Transp Syst 20(10):3806–3817

Jiang Y, Cao Z, Zhang J (2021) Learning to solve 3-D bin packing problem via deep reinforcement learning and constraint programming. IEEE Trans Cybernetics
Kadir BA, Broberg O, da Conceicao CS (2019) Current research and future perspectives on human factors and ergonomics in industry 4.0. Comput Ind Eng 137:106004
Kaelbling LP, Littman ML, Cassandra AR (1998) Planning and acting in partially observable stochastic domains. Artif Intell 101(1–2):99–134
Khalil E, Le Bodic P, Song L, Nemhauser G, Dilkina B (2016) Learning to branch in mixed integer programming. Proceedings of the AAAI Conference on Artificial Intelligence
Khalil E, Dai H, Zhang Y, Dilkina B, Song L (2017) Learning combinatorial optimization algorithms over graphs. Adv Neural Inform Proc Syst, 30
Kirkpatrick J, Pascanu R, Rabinowitz N, Veness J, Desjardins G, Rusu AA, Milan K, Quan J, Ramalho T, Grabska-Barwinska A (2017) Overcoming catastrophic forgetting in neural networks. Proc Natl Acad Sci 114(13):3521–3526
Kool W, Van Hoof H, Welling M (2018) Attention, learn to solve routing problems! *arXiv preprint arXiv:1803.08475*
Korte BH, Vygen J, Korte B, Vygen J (2011) Combinatorial optimization, vol 1. Springer
Kundu O, Dutta S, Kumar S (2019) Deep-pack: A vision-based 2d online bin packing algorithm with deep reinforcement learning. 2019 28th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)
LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521(7553):436–444. https://www.nature.com/articles/nature14539.pdf
Lee J, Lee Y, Kim J, Kosiorek A, Choi S, Teh YW (2019) Set transformer: A framework for attention-based permutation-invariant neural networks. International conference on machine learning
Lee S, Cho Y, Lee YH (2020) Injection mold production sustainable scheduling using deep reinforcement learning. Sustainability 12(20):8718
Li Y (2017) Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*
Li S, Jin X, Xuan Y, Zhou X, Chen W, Wang Y-X, Yan X (2019) Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. Adv Neural Inform Proc Syst, 32
Li K, Zhang T, Wang R, Wang Y, Han Y, Wang L (2021) Deep reinforcement learning for combinatorial optimization: Covering salesman problems. IEEE Trans Cybernetics
Li Y, Gu W, Yuan M, Tang Y (2022) Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network. Robot Comput Integr Manuf 74:102283
Lillicrap TP, Hunt JJ, Pritzel A, Heess N, Erez T, Tassa Y, Silver D, Wierstra D (2015) Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*
Lin C, Choy KL, Ho GT, Chung SH, Lam H (2014) Survey of green vehicle routing problem: past and future trends. Expert Syst Appl 41(4):1118–1138
Littman ML, Szepesvári C (1996) A generalized reinforcement-learning model: Convergence and applications. ICML, Vienna
Lowe R, Wu YI, Tamar A, Harb J, Abbeel P, O., Mordatch I (2017) Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems*, *30*
Lu Y, Huang X, Zhang K, Maharjan S, Zhang Y (2020) Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles. IEEE Trans Veh Technol 69(4):4298–4311
Manavalan E, Jayakrishna K (2019) A review of internet of things (IoT) embedded sustainable supply chain for industry 4.0 requirements. Comput Ind Eng 127:925–953
Martello S, Pisinger D, Vigo D (2000) The three-dimensional bin packing problem. Oper Res 48(2):256–267
Mazyavkina N, Sviridov S, Ivanov S, Burnaev E (2021) Reinforcement learning for combinatorial optimization: a survey. Comput Oper Res 134:105400
Mittal U, Panchal D (2023) AI-based evaluation system for supply chain vulnerabilities and resilience amidst external shocks: an empirical approach. Rep Mech Eng 4(1):276–289
Mittal U, Yang H, Bukkapatnam ST, Barajas LG (2008) Dynamics and performance modeling of multi-stage manufacturing systems using nonlinear stochastic differential Eq. 2008 IEEE International Conference on Automation Science and Engineering
Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G (2015) Human-level control through deep reinforcement learning. Nature 518(7540):529–533
Mnih V, Badia AP, Mirza M, Graves A, Lillicrap T, Harley T, Silver D, Kavukcuoglu K (2016) Asynchronous methods for deep reinforcement learning. International conference on machine learning
Munir MS, Abedin SF, Tran NH, Hong CS (2019) When edge computing meets microgrid: a deep reinforcement learning approach. IEEE Internet Things J 6(5):7360–7374

Nachum O, Norouzi M, Xu K, Schuurmans D (2017) Bridging the gap between value and policy based reinforcement learning. Adv Neural Inform Proc Syst, 30
Nazari M, Oroojlooy A, Snyder L, Takác M (2018) Reinforcement learning for solving the vehicle routing problem. Adv Neural Inform Proc Syst, 31
Pei Z, Yao X, Zhao W, Yu B (2023) Quantization via distillation and contrastive learning. IEEE Trans Neural Netw Learn Syst
Polydoros AS, Nalpantidis L (2017) Survey of model-based reinforcement learning: applications on robotics. J Intell Robotic Syst 86(2):153–173
Puche AV, Lee S (2022) Online 3D Bin Packing Reinforcement Learning Solution with Buffer. *arXiv preprint arXiv:2208.07123*
Qiu D, Wang Y, Zhang T, Sun M, Strbac G (2022) Hybrid multi-agent reinforcement learning for Electric Vehicle Resilience Control towards a low-Carbon Transition. IEEE Trans Industr Inf
Salvendy G (2001) Handbook of industrial engineering: technology and operations management. Wiley, Hoboken
Sbihi A, Eglese RW (2010) Combinatorial optimization and green logistics. Ann Oper Res 175(1):159–175
Sgarbossa F, Grosse EH, Neumann WP, Battini D, Glock CH (2020) Human factors in production and logistics systems of the future. Annu Rev Control 49:295–305
Shannon RE, Long SS, Buckles BP (1980) Operation research methodologies in industrial engineering: a survey. AIIE Trans 12(4):364–367
Shi J, Gao Y, Wang W, Yu N, Ioannou PA (2019) Operating electric vehicle fleet for ride-hailing services with reinforcement learning. IEEE Trans Intell Transp Syst 21(11):4822–4834
Silver D, Schrittwieser J, Simonyan K, Antonoglou I, Huang A, Guez A, Hubert T, Baker L, Lai M, Bolton A (2017) Mastering the game of go without human knowledge. Nature 550(7676):354–359
Smith KA (1999) Neural networks for combinatorial optimization: a review of more than a decade of research. Informs J Comput 11(1):15–34
Spaan MT (2012) Partially observable Markov decision processes. Reinforcement learning. Springer, Cham, pp 387–414
Su W, Li Z, Xu M, Kang J, Niyato D, Xie S (2024) Compressing deep reinforcement learning networks with a dynamic structured pruning method for autonomous driving. IEEE Trans Veh Technol
Sun Y, Tan W (2019) A trust-aware task allocation method using deep q-learning for uncertain mobile crowdsourcing. Human-centric Comput Inform Sci 9(1):25. https://doi.org/10.1186/s13673-019-0187-4
Sutton RS (1984) Temporal credit assignment in reinforcement learning. University of Massachusetts Amherst, Amherst
Sutton RS (1988) Learning to predict by the methods of temporal differences. Mach Learn 3(1):9–44
Sutton RS, Barto AG (2018) Reinforcement learning: an introduction. MIT Press, Cambridge
Sutton RS, McAllester D, Singh S, Mansour Y (1999) Policy gradient methods for reinforcement learning with function approximation. Adv Neural Information processing systems, 12
Tao Y, Qiu J, Lai S (2022) A Learning and Operation Planning Method for Uber Energy Storage System: Order Dispatch. IEEE Trans Intell Transp Syst
Tavakoli A, Pardo F, Kormushev P (2018) Action branching architectures for deep reinforcement learning. Proceedings of the AAAI Conference on Artificial Intelligence
Toth P, Vigo D (2002) Models, relaxations and exact approaches for the capacitated vehicle routing problem. Discrete Appl Math 123(1–3):487–512
Triantaphyllou E, Mann SH (1995) Using the analytic hierarchy process for decision making in engineering applications: some challenges. Int J Industrial Engineering: Appl Pract 2(1):35–44
Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Adv Neural Inform Proc Syst, 30
Vázquez-Canteli JR, Nagy Z (2019) Reinforcement learning for demand response: a review of algorithms and modeling techniques. Appl Energy 235:1072–1089
Verma R, Singhal A, Khadilkar H, Basumatary A, Nayak S, Singh HV, Kumar S, Sinha R (2020) A generalized reinforcement learning algorithm for online 3d bin-packing. *arXiv preprint arXiv:2007.00463*
Vinyals O, Bengio S, Kudlur M (2015a) Order matters: Sequence to sequence for sets. *arXiv preprint arXiv:1511.06391*
Vinyals O, Fortunato M, Jaitly N (2015b) Pointer networks. Adv Neural Inform Proc Syst, 28
Wan Z, Li H, He H, Prokhorov D (2018) Model-free real-time EV charging scheduling based on deep reinforcement learning. IEEE Trans Smart Grid 10(5):5246–5257
Wang Z, Gombolay M (2020) Learning scheduling policies for multi-robot coordination with graph attention networks. IEEE Rob Autom Lett 5(3):4509–4516
Wang Z, Schaul T, Hessel M, Hasselt H, Lanctot M, Freitas N (2016) Dueling network architectures for deep reinforcement learning. International conference on machine learning

Wang L, Hu X, Wang Y, Xu S, Ma S, Yang K, Liu Z, Wang W (2021) Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning. Comput Netw 190:107969
Williams RJ (1992) Simple statistical gradient-following algorithms for connectionist reinforcement learning. Mach Learn 8(3):229–256
Wolpert DH, Macready WG (1997) No free lunch theorems for optimization. IEEE Trans Evol Comput 1(1):67–82
Xie R, Zhang S, Wang R, Xia F, Lin L (2021) Explore, filter and distill: Distilled reinforcement learning in recommendation. Proceedings of the 30th ACM International Conference on Information & Knowledge Management
Yan Y, Chow AH, Ho CP, Kuo Y-H, Wu Q, Ying C (2022) Reinforcement learning for logistics and supply chain management: methodologies, state of the art, and future opportunities. Transp Res E 162:102712
Yang T, Zhao L, Li W, Zomaya AY (2020) Reinforcement learning in sustainable energy and electric systems: a survey. Annu Rev Control 49:145–163
Yu L, Zhang C, Jiang J, Yang H, Shang H (2021) Reinforcement learning approach for resource allocation in humanitarian logistics. Expert Syst Appl 173:114663
Zhang K, Leng S, Peng X, Pan L, Maharjan S, Zhang Y (2018) Artificial intelligence inspired transmission scheduling in cognitive vehicular communications and networks. IEEE Internet Things J 6(2):1987–1997
Zhang C, Liu Y, Wu F, Tang B, Fan W (2020a) Effective charging planning based on deep reinforcement learning for electric vehicles. IEEE Trans Intell Transp Syst 22(1):542–554
Zhang Z, Zhou Y, Ji X (2020b) Almost optimal model-free reinforcement learningvia reference-advantage decomposition. Adv Neural Inf Process Syst 33:15198–15207
Zhang R, Lv Q, Li J, Bao J, Liu T, Liu S (2022a) A reinforcement learning method for human-robot collaboration in assembly tasks. Robot Comput Integr Manuf 73:102227
Zhang Y, Bai R, Qu R, Tu C, Jin J (2022b) A deep reinforcement learning based hyper-heuristic for combinatorial optimisation with uncertainties. Eur J Oper Res 300(2):418–427
Zhao J, Mao M, Zhao X, Zou J (2020) A hybrid of deep reinforcement learning and local search for the vehicle routing problems. IEEE Trans Intell Transp Syst 22(11):7208–7218
Zhao H, Zhu C, Xu X, Huang H, Xu K (2022) Learning practically feasible policies for online 3D bin packing. Sci China Inform Sci 65(1):1–17
Zhou T, Tang D, Zhu H, Zhang Z (2021) Multi-agent reinforcement learning for online scheduling in smart factories. Robot Comput Integr Manuf 72:102202

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.